

CodeS+: Towards Assessing the Generalization Ability of Code Models Under Distribution Shift

Ziyue Shi , Junjie Wang , Yuejun Guo , Xiaofei Xie , *Member, IEEE*, Qiang Hu , Maxime Cordy ,
Sen Chen , Mike Papadakis , Yves Le Traon , *Fellow, IEEE*, and Yongqiang Lyu 

Abstract—Distribution shift poses a significant challenge for deep learning (DL) models in source code analysis, where test data often follows different distributions from training data, leading to unexpected performance degradation and hindering the practical usage of code models. While our previous work CodeS introduced the first benchmark for studying distribution shift in source code analysis, it has limitations in covering more fine-grained types of real-world distribution shifts and lacks the study of the effectiveness of shift mitigation strategies. In this paper, we present CodeS+, an enhanced benchmark that addresses these limitations through two key contributions, (1) expanded shift types, we design more fine-grained distribution shift types, that is, shift introduced by different program element complexity (e.g., different node number of control flow graphs), and (2) investigate the usefulness of fine-tuning based shift mitigation techniques, such as Core-Set. Comprehensive experiments on different pre-trained code models demonstrated that code models significantly suffer from distribution shift, out-of-distribution detectors from other domains (e.g., computer vision) do not generalize to source code, and existing fine-tuning based shift mitigation techniques have limited benefits in enhancing the generalization ability of code models. Our findings highlight the need to pay more attention to OOD issues for code models.

Index Terms—Source code learning, distribution shift, out-of-distribution detection, model generalization.

Received 15 August 2025; revised 14 February 2026; accepted 18 February 2026. Date of publication 12 March 2026; date of current version 20 April 2026. This work was supported by the National Natural Science Foundation of China under Grant U24A6009. Recommended for acceptance by C. McMillan. (Ziyue Shi and Junjie Wang are contributed equally to this work.) (Corresponding author: Qiang Hu.)

Ziyue Shi, Junjie Wang, and Qiang Hu are with Tianjin University, Tianjin 300100, China (e-mail: ziyueshi0723@tju.edu.cn; junjie.wang@tju.edu.cn; qianghu@tju.edu.cn).

Yuejun Guo is with Luxembourg Institute of Science and Technology, Belval L-1359, Luxembourg (e-mail: yuejun.guo@list.lu).

Xiaofei Xie is with Singapore Management University, Singapore 188065 (e-mail: xiaofei.xfxie@gmail.com).

Maxime Cordy, Mike Papadakis, and Yves Le Traon are with the University of Luxembourg, Belval L-1359, Luxembourg (e-mail: maxime.cordy@uni.lu; michail.papadakis@uni.lu; Yves.LeTraon@uni.lu).

Sen Chen is with Nankai University, Tianjin 300100, China (e-mail: senchen@nankai.edu.cn).

Yongqiang Lyu is with the Tianjin University, China, and also with the Joint Research Center for System Security, Tsinghua University (Institute for Network Sciences and Cyberspace)—Science City (Guangzhou) Digital Technology Group Company Ltd., Beijing 100084, China (e-mail: lyuyq@tju.edu.cn).

Digital Object Identifier 10.1109/TSE.2026.3668096

I. INTRODUCTION

THE advent of deep learning (DL) has fundamentally reshaped the field of source code analysis. Traditional techniques, which often require static rules and manual feature engineering, have proven increasingly inadequate for handling the complexity and scale of modern software systems. In contrast, DL models offer remarkable capabilities in automatically discerning intricate patterns and features from a vast code corpus. This paradigm shift has enabled significant advances across various code analysis tasks, such as source code classification [1], software vulnerability detection [2], and code summarization [3]. Particularly, recent reports [4] highlight that large language models have achieved competitive and even better results than developers.

As these DL models for source code analysis transition from research prototypes to production systems, they face a critical and persistent challenge: the gap between controlled experimental performance and real-world reliability. This gap is especially evident when models encounter distribution shift [5], where test data in deployment differs from training distributions, leading to unexpected performance degradation. In practice, this shift manifests in multiple dimensions [6]—code may originate from different programming paradigms, authored by developers with differing coding styles, or reflect temporal changes in programming practices. Such a shift makes a substantial impact on model performance and generalization.

The broader machine learning community, e.g., computer vision [5] and natural language processing [7], has made remarkable progress in creating benchmarks and addressing distribution shift challenges. For example, CIFAR-10-C [8] provides 15 types of algorithmically generated corruptions that generate images with different distributions from CIFAR-10. Wilds [5] collects natural datasets with distribution shifts including both image and text data in the wild. However, the distribution shift study in the field of source code analysis is still in its infancy.

Our previous work, CodeS [6], addressed this gap by introducing a comprehensive benchmark for evaluating the generalization ability of DL models under distribution shifts in source code analysis tasks. The benchmark established five key shift types—task, programmer, time, token, and Concrete Syntax Tree (CST) shifts—across Java and Python programming languages. CodeS demonstrated that distribution shift poses significant challenges for source code analysis models, revealing substantial performance degradation across different shift scenarios.

Despite its contributions, CodeS suffers from several important limitations that constrain its applicability to real-world scenarios. First, CodeS is constrained to competition-level programming tasks sourced from programming competition platforms, which may not adequately represent the complexity and diversity of real-world software development scenarios. Second, CodeS employs a relatively coarse-grained approach to measure distribution shifts. For example, CodeS measures the distribution shift by simply leveraging the similarity between code snippets, thus lacking a fine-grained distribution shift definition to help better understand the capability of code models. Third, the current framework of CodeS is primarily focused on evaluation and analysis without systematically investigating the effectiveness of shift mitigation techniques, such as model retraining, which would provide actionable guidance for researchers and developers seeking to improve model generalization under distribution shift.

In this paper, we present CodeS+, an enhanced benchmark dataset that substantially extends the scope and utility of distribution shift evaluation for source code analysis. Motivated by the growing emphasis in code model research [9], [10], [11], which increasingly demonstrates that incorporating program graphs significantly enhances model comprehension of code semantics, underscoring the critical role of structured representations, we introduce two novel shift categories, node distribution shift and edge distribution shift to tackle the limitation of CodeS which lacks fine-grained distribution shift definitions. Concretely, these two shift types quantify the structural complexity of programs using the node number and edge number of graphs (e.g., data flow graph) extracted from these programs. Based on the new definitions, we extend CodeS to support fine-grained distribution shifts and construct CodeS+. Using CodeS+, we conduct experiments to investigate 1) the generalization ability of existing source code models, 2) the effectiveness of existing OOD detectors in distinguishing ID and OOD source code data, and 3) the usefulness of fine-tuning-based OOD mitigation techniques in enhancing the generalization ability of code models. Experimental results demonstrated that code models suffer significantly from the shift in data distribution, and existing OOD detectors and OOD mitigation techniques have limited benefits in handling distribution shift for code data.

To summarize, the main contributions of this paper are:

- We propose CodeS+¹, an extension of CodeS that provides fine-grained distribution shift datasets to support the new evaluation paradigm (generalization ability evaluation) for code models.
- Using CodeS+, we reveal that code models suffer from distribution shift, i.e., distribution shifts cause significant degradation in model performance.
- We perform a comprehensive evaluation of the effectiveness of existing OOD detectors and OOD mitigation techniques using CodeS+, and highlight their limitation in handling code data.

¹Please visit our companion site for more details (data, model, implementation, and results): <https://sites.google.com/view/codes-distribution-shift>.

The structural organization of this paper proceeds as follows. Section II introduces the background. Section III presents the original CodeS and the enhanced CodeS+. Section IV describes the experimental design. Section V presents the results, followed by the discussion in Section VI. Section VII overviews the related work. The last section concludes this paper.

II. BACKGROUND

A. Machine Learning for Code

The transformative success of pre-trained natural language models such as BERT [12], evidenced by breakthroughs in NLP tasks such as sentiment analysis [13], [14] has directly catalyzed a paradigm shift in machine learning for code. This shift redefines machine learning methodologies from syntax-bound feature engineering (e.g., Code2Vec's [15] reliance on abstract syntax tree paths) toward contextually rich semantic understanding, driving the creation of pre-trained code models [16], [17], [18], [19] that significantly boost performance across programming language processing tasks.

This evolution is anchored in self-supervised machine learning frameworks adapted from natural language processing, where models like CodeBERT [20] and GraphCodeBERT [21] leverage massive bimodal datasets (e.g., CodeSearchNet [22]) to capture cross-modal relationships between code and natural language. Feng et al. proposed CodeBERT that shares the same model architecture as RoBERTa [23]. CodeBERT employs dual training objectives, masked language modeling (MLM) to recover masked tokens and replaced token detection (RTD) to enable learning from unimodal code data which yields robust representations for classification tasks. GraphCodeBERT extends this by explicitly integrating data-flow graphs (DFG) through node alignment and edge prediction tasks, enabling structure-aware embeddings that achieve high accuracy.

While pre-trained code models like CodeBERT and GraphCodeBERT represent the vanguard of ML4Code with their contextual semantic understanding, classical DNN paradigms such as MLP and CNN provided by the Project CodeNet [24] retain enduring methodological relevance. The inclusion of MLP (Bag), capturing global statistical regularities through bag-of-tokens representations, and CNN (Sequence), extracting hierarchical syntactic patterns via convolutional filters. In this paper, we focus on analyzing MLP, CNN, RoBERTa, CodeBERT and GraphCodeBERT as these models have representative significance in ML4code.

B. Distribution Shift

Distribution shift, defined as the divergence between the training and test data distributions (examples are shown in Fig. 1), constitutes a fundamental challenge in machine learning domains like computer vision [5] and natural language processing [7]. This phenomenon in source code manifests itself through two primary modalities: synthetic shift induced by artificial perturbations (e.g., adversarial code transformations mimicking real-world obfuscation) and natural shift arising from organic variations like temporal evolution or contextual discrepancies.

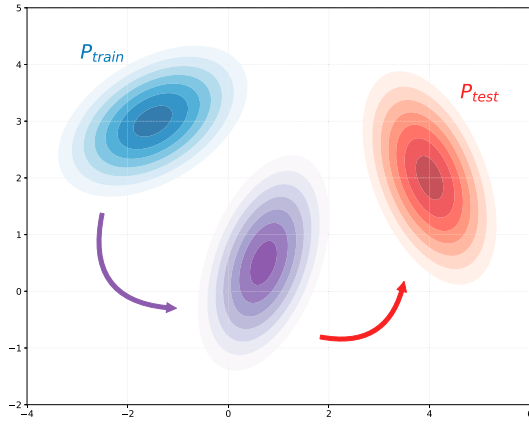


Fig. 1. An example of distribution shifts.

However, in the domain of source code intelligence, the systematic study of distribution shifts remains nascent and fragmented. Benchmarks like CIFAR-10-C [8] and Wilds [5] have established rigorous evaluation frameworks for CV and NLP, encompassing both synthetic corruptions (e.g., noise, blur) and natural shifts (e.g., geographic or temporal variations), equivalent standards for code data are critically lacking. Existing efforts [5], [25] primarily address simplistic scenarios, such as cross-project migration or coarse-grained temporal drift (e.g., training on legacy Java projects and testing in modern ones).

In this paper, we further extend the original CodeS [6] dataset by incorporating new types of distribution shifts, with particular focus on graph structural differences. Detailed descriptions are provided in Section III-B.

III. CODES+: ENHANCED CODES BENCHMARK

A. Overview of CodeS

CodeS [6] provides a benchmark dataset for evaluating the generalization of DL models in source code analysis under distribution shift. It includes 16 groups of in/out-of-distribution (ID/OOD) code data written in 2 programming languages, Java and Python, from five distinct shift categories:

- 1) **Task distribution shift:** ID data and OOD data target different tasks. This shift evaluates cross-task generalization where models trained to identify code patterns across multiple programming tasks (e.g., various graph algorithms, dynamic programming problems, or string manipulation tasks) are tested on code from entirely different algorithmic domains.
- 2) **Programmer distribution shift:** ID data and OOD data target the same task but are authored by different programmers. Due to the programming habit, the distribution shift can happen across different programmers.
- 3) **Time distribution shift:** ID code and OOD code target the same task but are written in different time periods. Assessing [26] and maintaining [27] the model performance over time is critical to ensure the reliability and security of models given that the code can be modified or new users appear over time.

- 4) **Token distribution shift:** ID code and OOD code target the same task but the frequencies of tokens that appear to them are different. In source code learning, transforming code into numerical vectors is fundamental to make it executable for deep learning models [28]. A linear sequence of tokens is the typical and most important code representation, which is usually processed via tokenization or lexical analysis. A token is the basic unit of the representation and can be a function name, an operator, or a punctuation sign. In practice, each token is represented by an integer to be compatible with models. Given the token sequences of two source code files, the straightforward difference is the appearance of tokens.

- 5) **Concrete Syntax Tree (CST) distribution shift:** ID code and OOD code target the same task but have different CST representations. The concrete syntax tree (also known as the parse tree or derivation tree) is another popular code representation, which includes the syntactic structure of code files. The difference between code files can be represented by the distance between CSTs.

Raw data in Codes. The data in CodeS comes from different sources. The raw data of Python75 is collected from the AtCoder site [29], where new submissions and programmers are continuously announced in the contests. We select the submissions from 75 tasks in 25 contests based on the following conditions: 1) the language should be Python3, 2) the tasks have at least 1,000 submissions, and 3) the status of the submission is accepted (AC). In total, we collect 200,462 programs for Python75. The raw data of Java250-S and Python800-S are from Java250 and Python800 provided by Project CodeNet [24]. In total, Java250-S and Python800-S contain 75000 and 240000 programs.

While CodeS provides foundational insights into distribution shift challenges, it primarily focuses on characterizing the problem rather than developing systematic mitigation approaches. This limitation motivates our enhanced CodeS+ framework, which extends the original benchmark with additional shift types and introduces effective mitigation strategies.

B. New Shift Types: Motivation and Design

Motivation: There is a significant limitation in research on the robustness of distribution shift in current machine learning models for source code analysis tasks: existing works on distribution shift in source code mainly focus on the semantic syntax characteristics of code or other continuous or discrete data distributions, but lack targeted modeling of structured program representations.

Considering the particularity of source code data, such as syntax tree hierarchy and control flow dependencies, it is required to construct shift types that can quantify structural complexity. To this end, we further propose distribution shifts based on the multi-graph perspective of source code. By parsing the CPGs of the source code, we systematically define edge distribution shift and node distribution shift, which reflect the topological changes of the program structure (such as the number of nodes/edges) and capture semantic differences between

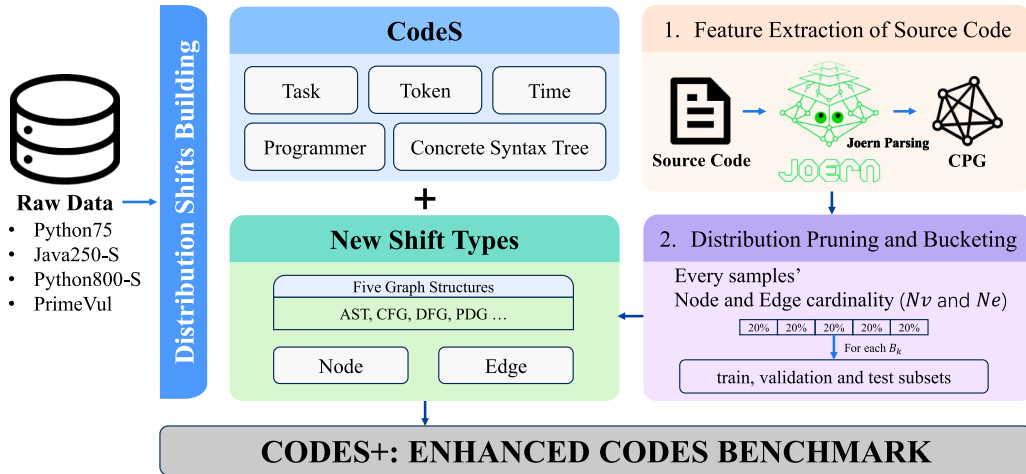


Fig. 2. The building pipeline of new distribution shifts.

source code. This design fills the gap in the formal definition of distribution offset in the field of code analysis and provides a more comprehensive benchmark for evaluating the generalization ability of models.

Based on the established benchmark in CodeS [6], we further extend this framework by introducing two novel shift categories:

- 1) **Node distribution shift:** ID code and OOD code target the same task but exhibit divergent structural complexity quantified by node cardinality in equivalent graph representations of source code.
- 2) **Edge distribution shift:** ID code and OOD code target the same task but exhibit divergent structural complexity quantified by edge cardinality in equivalent graph representations of source code.

For each of the above-mentioned distribution shifts, we consider five categories of graph structures derived from source code: Abstract Syntax Tree (AST), Control Flow Graph (CFG), Data Flow Graph (Dataflow), Program Dependency Graph (PDG), and Reference Type Graph (ReferType). Based on the edge cardinality or node cardinality per sample, datasets are further partitioned into ID/OOD sets, each with a training, validation, and test set.

Specifically, the construction workflow of edge and node distribution shifts is shown in Fig. 2, which can be primarily divided into two stages:

- 1) **Feature Extraction of Source Code:** Given a raw dataset, we parse each sample in the dataset into a unified Code Property Graph (CPG) [30] representation using the Joern static analysis tool [31]. Then, the CPG of each sample is decoupled into five distinct graph structures as mentioned before. For each graph type, core topological features are extracted: the number of nodes and the number of edges. To ensure feature validity, samples with parsing failures (e.g., unsupported syntax) are excluded.
- 2) **Distribution Pruning and Bucketing:** Then, for each graph type, we perform quantile-based pruning to remove samples that have extremely few or many nodes and

TABLE I
DATA INFORMATION OF NODE/EDGE DISTRIBUTION SHIFT

Shift type	Cardinality Range			
	Python75	Java250-S	Python800-S	PrimeVul
Node				
AST	34-199	36-130	43-319	21-147
CFG	24-151	27-108	31-247	14-122
Dataflow	34-199	36-131	43-319	21-147
PDG	23-144	28-105	30-238	17-124
Reftype	35-188	31-113	44-303	21-125
Edge				
AST	34-200	43-139	43-321	21-147
CFG	23-153	27-108	30-251	14-129
Dataflow	57-353	70-247	73-573	35-275
PDG	42-359	56-301	59-616	38-395
Reftype	39-244	37-149	51-395	23-158

edges. Mention that, even though this step could break the original distribution of datasets, our goal is more than that, to prepare a new one that reflects diverse data distribution to support the comprehensive evaluation of code models. After the pruning, the remaining data samples are equally divided into five groups based on the cumulative distribution of node numbers or edge numbers. The distributions of node and edge are summarized in Table I. As a result, given a dataset, for each graph type, CodeS+ contains 10 groups of data, where five groups are prepared for the *Node distribution shift* according to the complexity of nodes, and the others are curated for the *Edge distribution shift* based on the complexity of edges. In each shift, one group can be treated as in-distribution, and the others are datasets with distribution shift. For instance, the Python75 dataset under node distribution shift of the AST graph type is divided into five distinct groups, each corresponding to specific percentile intervals of the entire data based on node numbers: the first 20% (0–20%), the second 20% (20–40%), the third 20% (40–60%), and so on.

C. Dataset Creation

Building upon the CodeS framework, we extend the datasets by applying our edge and node distribution shifts to generate new benchmark datasets from their original raw data: Java250-S, Python800-S, and Python75. Additionally, we introduce Primevul [32], a high-quality vulnerability detection dataset designed to provide a more realistic evaluation benchmark for Code Language Models (Code LMs) in software vulnerability detection tasks. The dataset comprises 6,968 vulnerable function samples and 228,800 benign function samples. All data sources originate from real-world C/C++ projects, significantly enhancing the reliability of model generalization capability assessment. In CodeS+, we further split PrimeVul based on node/edge cardinality to build distribution shifts similar to the others.

All datasets (Java250-S, Python800-S, Python75, and Primevul) undergo standardized pre-processing through the edge/node distribution shift workflow illustrated in Fig. 2. In order to avoid the performance degradation caused by truncation interfering with the results of distribution shift, we conducted a further analysis on datasets (Java250-S, Python800-S, Python75). The proportion of examples that needed to be truncated in each group of each dataset was less than one percent. Moreover, no correlation was found between the proportion and the increase in graph complexity. For Primevul, we implement three additional critical enhancements to address domain-specific challenges: 1) we filter samples with fewer than three executable lines and all comment blocks in the code to eliminate irrelevant samples lacking measurable structural complexity. 2) Using paired vulnerable/benign function samples, we identify line-level vulnerability locations via differential code analysis and integrate these annotations into a sliding activity window during model training. This ensures explicit exposure to security-critical code patterns when learning edge/node distribution shifts, avoiding the impact caused by truncation. 3) We use the commit URL in the Primevul dataset to crawl the commit timestamp from the source repository to build the time distribution shift introduced by CodeS. In addition, since the data in the PrimeVul datasets originates from real-world vulnerabilities, there exists an extreme imbalance between positive and negative samples. To mitigate potential confounding effects, we maintain a consistent 9:1 ratio of negative to positive samples across all data groups during the construction of distribution shifts in PrimeVul. These analyses and processes ensure that our distribution shifts exhibit minimal correlation with factors related to length or pure syntactic complexity. Tables II, III summarize the characteristics of CodeS+. Table IV lists the overview of the comparison between existing datasets and CodeS+. We can see CodeS+ covers the most diverse distribution shift types. Fig. 3 shows some concrete examples in CodeS+, more details can be found on our project site¹.

IV. EXPERIMENTAL SETUP

Based on CodeS+, we investigate the generalization ability of widely used code models, the effectiveness of OOD detection methods, and shift mitigation techniques. In subsequent results

TABLE II
DATA INFORMATION (NUMBER OF CLASSES, DATA SIZE PER CLASS)

Data Collection	Programmer, Time, Token, CST (Task)			
	#ID/OOD Classes	#Training	#ID Test	#OOD Test
Python75	75/75 (65/10)	732 (846)	134 (154)	134 (1000)
Java250-S	250/250 (200/50)	180 (225)	60 (75)	60 (300)
Python800-S	800/800 (640/160)	180 (225)	60 (75)	60 (300)
PrimeVul	2/2 (2/2)	4500/500	900/100	900/100

TABLE III
DATA INFORMATION (NUMBER OF CLASSES, DATA SIZE PER GROUP)

Data Collection	Node, Edge			
	#ID/OOD Classes	#Training	#ID Test	#OOD Test
Python75	75/75	10000	1250	1250
Java250-S	250/250	5000	625	625
Python800-S	800/800	10000	625	625
PrimeVul	2/2	5000	1000	1000

TABLE IV
COMPARISON BETWEEN OUR DEVELOPED CODES+ AND EXISTING DATASETS

	Programming Language	Task	Programmer	Time-Stamp	Token	CST	Node	Edge
Wilds [5]	Python	✓	✗	✗	✗	✗	✗	✗
[25]	Java	✓	✓	✓	✗	✗	✗	✗
[33]	Java	✓	✗	✓	✗	✗	✗	✗
CodeS+	Python, Java	✓	✓	✓	✓	✓	✓	✓

of Node or Edge distribution shifts, the ID data is defined as the first 20% of samples, while the OOD data comprises the next 20% (20-40%). Our in-depth evaluation and analysis answer the following research questions:

- **RQ1:** How do the code models perform in the face of data distribution shift?
- **RQ2:** How effective are OOD detectors in detecting distribution shift introduced by CodeS+?
- **RQ3:** Can existing OOD mitigation methods alleviate the distribution shift caused by CodeS+?

A. Deep Neural Networks

We consider six distinct deep neural network architectures for representing code. The first two models, a Convolutional Neural Network (CNN) featuring dual max and average global pooling (doublePoolClassDNN) and a Multi-Layer Perceptron (MLP) based on a bag-of-words approach (denseDNN), are sourced from the Project CodeNet [24]. We incorporate three pre-trained bimodal models: RoBERTa [23], CodeBERT [20], and GraphCodeBERT [21], initialized using publicly released weights from the CodeXGLUE project [34]. Additionally, we include a pre-trained large language model (LLM) StarCoder2-3B [35] released by the BigCode project. Each of these pre-trained models undergoes fine-tuning on our training dataset. Note that these models differ in size, architecture, and fine-tuning method; our goal is to assess distribution shifts broadly across model types rather than isolate individual factors. We acknowledge that using larger LLMs (7B+) may yield different robustness characteristics, which we leave as future work due to computational constraints.

Data type: ID (0-20%) NODE: AST with 36 nodes Program: <pre>import java.util.*; class Main { public static void main(String[] args) { Scanner sc = new Scanner(System.in); String s = sc.next(); boolean ok = s.matches("(dream(er)?(eraser?))"); System.out.println(ok?"YES":"NO"); } }</pre>	Data type: OOD (20-40%) NODE: AST with 70 nodes Program: <pre>import java.util.*; public class Main { public static void main(String[] args) { Scanner sc = new Scanner(System.in); String S = sc.next(); S = S.replaceAll("eraser", "-"); S = S.replaceAll("dreamer", "-"); S = S.replaceAll("dream", "-"); S = S.replaceAll("dream", "-"); S = S.replaceAll("dream", "-"); if(S.length() == 0) System.out.println("YES"); else System.out.println("NO"); } }</pre>
--	--

(a) Distribution Type: Node-AST

Data type: ID (0-20%) EDGE: AST with 46 edges Program: <pre>import java.util.*; import java.math.BigDecimal; public class Main { public static void main(String args[]) { Scanner sc = new Scanner(System.in); BigDecimal A = sc.nextBigDecimal(); BigDecimal B = sc.nextBigDecimal(); BigDecimal ans = A.multiply(B); System.out.println(ans.toBigInteger()); } }</pre>	Data type: OOD (20-40%) EDGE: AST with 81 edges Program: <pre>import java.util.*; public class Main { public static void main(String[] args) { Scanner sc = new Scanner(System.in); long A = sc.nextLong(); String B = sc.next(); sc.close(); int B100 = 0; B100 += Integer.parseInt(B.substring(0, 1)) * 100; B100 += Integer.parseInt(B.substring(2, 3)) * 10; B100 += Integer.parseInt(B.substring(3, 4)) * 1; System.out.println((long) A * B100 / 100); } }</pre>
--	--

(b) Distribution Type: Edge-AST

Fig. 3. Examples in CodeS+ under node/edge distribution shifts (Dataset: Java250-S).

B. OOD Detectors

An OOD detector is a method that, given a trained model, distinguishes between ID and OOD data at inference time. We employ four widely used OOD detection methods that have been extensively evaluated across various domains and model architectures [36]. The first three are output (softmax) score-based and do not require any change in the model.

Let $x \in \mathcal{X}$ denote an input sample, and $y \in \mathcal{Y}$ the corresponding class label, where $\mathcal{Y} = \{1, 2, \dots, C\}$ for a classification task with C classes. Let $f(x)$ be the feature representation of x , and $z = f_\theta(x) \in \mathbb{R}^C$ be the output logits of a neural network parameterized by θ . The softmax probability is denoted as $p(y | x)$.

Maximum Softmax Probability (MSP) [37] is the simplest OOD detector. It uses the maximum value of the softmax output as the confidence score. Assuming that a well-trained model should produce a higher confidence score for ID data than OOD data, MSP detects OOD data by thresholding:

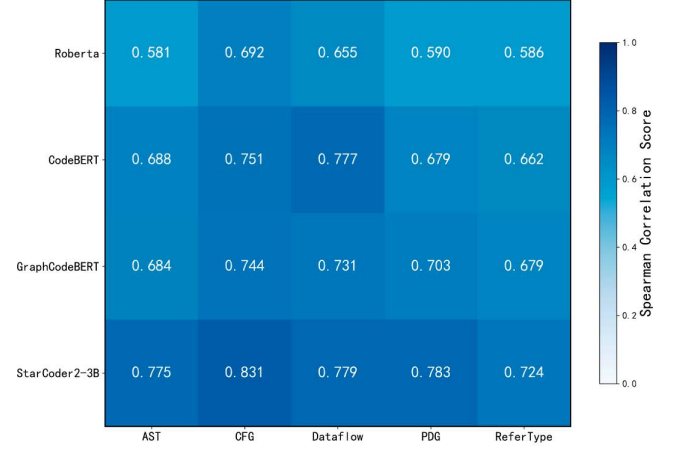
$$\text{MSP}(x) = \max_{y \in \mathcal{Y}} p(y | x). \quad (1)$$

Data with low $\text{MSP}(x)$ is identified as OOD.

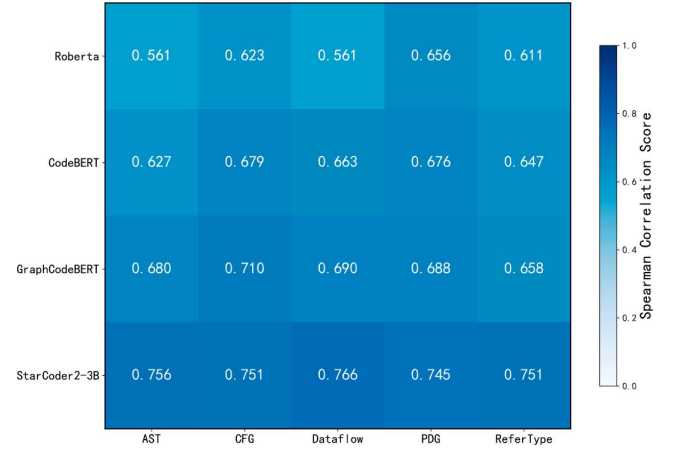
Out-of-Distribution detector for neural networks (ODIN) [38] improves upon MSP by applying temperature scaling and small adversarial perturbations to the input to amplify the difference between ID and OOD data. The softmax output is defined as:

$$p(y | x; T) = \frac{\exp(z_y/T)}{\sum_j \exp(z_j/T)} \quad (2)$$

where $T > 1$ is the temperature to sharpen the softmax distribution.



(a) Edge



(b) Node

Fig. 4. The Spearman correlation between model confidence and correctness on OOD.

Mahalanobis distance-based detection [39] computes the confidence score based on an induced generative classifier under Gaussian discriminant analysis (GDA) instead of relying on the softmax outputs. The confidence score is defined using the Mahalanobis distance between a test data and its closest class-conditional Gaussian distribution:

$$\text{M}(x) = \max_c - (f(x) - \mu_c)^\top \Sigma^{-1} (f(x) - \mu_c) \quad (3)$$

where μ_c is the mean of class c , and Σ is the shared covariance matrix. Data with low $\text{M}(x)$ is considered as ID.

Outlier Exposure (OE) [40] differs from the previous methods by modifying the training procedure. OE trains a new neural network using both ID and OOD data, using the same architecture as the given model. The new model is trained with a composite loss function:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{ID}} + \lambda \cdot \mathcal{L}_{\text{OE}} \quad (4)$$

where $\mathcal{L}_{\text{ID}}$ is the standard classification loss on ID data, and $\mathcal{L}_{\text{OE}}$ encourages low-confidence predictions for OOD data. This

explicit exposure during training enables the model to better distinguish ID from OOD samples at test time.

Feature-based Likelihood Ratio Score (FLatS) [41] introduces a principled approach to OOD detection by leveraging the likelihood ratio between ID and OOD probabilities. FLatS defines the OOD score as the difference between energy functions estimated in the feature space, where the likelihood ratio is approximated as:

$$S_{\text{FLatS}}(x) = E_{\text{id}}(x; ID) - \alpha \cdot E_{\text{ood}}(x; OOD) \quad (5)$$

where E is the feature-based likelihood ratio of sample x with respect to ID or OOD data, α is a scaling hyper-parameter to make the two scores comparable.

C. OOD Mitigation

OOD mitigation strategies aim to enhance model robustness against distribution shifts. Following previous works [27], [42], [43], we consider training-based OOD mitigation techniques, which are the most effective ones. Concretely, we employ four representative sampling-based mitigation techniques, which prioritize samples that are more useful for enhancing the robustness of models and then fine-tune models accordingly.

Entropy-based Mitigation [44] Entropy quantifies the uncertainty of model predictions by measuring the information dispersion in softmax outputs. For input x , the predictive entropy is defined as

$$H(x) = - \sum_{c=1}^C p(y_c|x) \log_2 p(y_c|x) \quad (6)$$

where $p(y_c|x)$ denotes the softmax probability for class c . Higher entropy indicates flatter probability distributions, characteristic of OOD samples. The selection function identifies samples with maximal entropy:

$$\text{select_index} = \text{argsort}(H(x)) [0 : K] \quad (7)$$

where K is the selection size. This method directly targets semantic shifts where models exhibit ambiguous predictions across all classes.

Gini Coefficient-based Mitigation [42] The Gini coefficient evaluates prediction skew by computing the purity of softmax distributions:

$$G(x) = 1 - \sum_{c=1}^C p(y_c|x)^2 \quad (8)$$

Lower Gini scores (approaching 0) indicate uniform distributions (typical of OOD data), while higher scores (approaching 1) reflect confident predictions (ID data). The selection prioritizes minimal Gini scores:

$$\text{select_index} = \text{argsort} \left(\sum_{c=1}^C p(y_c|x)^2 \right) [0 : K] \quad (9)$$

Gini scale invariance makes it robust to class imbalance, avoiding bias from head classes in long-tailed distributions.

Core-set based Mitigation [45] covariates shifts by identifying geometrically representative subsets. Let $F = f(x_i)$ denote

feature embeddings of unlabeled data. The selection uses a Min-Max distance criterion:

$$\text{select_index} = \underset{S \subseteq U}{\text{argmax}} \left(\min_{z \in \mathcal{L} \cup S} |f(x) - f(z)|_2 \right) [0 : K] \quad (10)$$

where $\mathcal{L}$ is the labeled set (randomly selected OOD samples) and U the unlabeled set (entire OOD samples). The algorithm iteratively selects samples maximizing the minimum distance to existing labeled prototypes, ensuring the coverage of feature space. This isolates shifts where input statistics deviate, but semantic boundaries persist.

Random Mitigation serves as a baseline by stochastically sampling data:

$$\text{select_index} \sim \text{Uniform}(0, N), \quad |\text{select_index}| = K \quad (11)$$

Random selection method uniformly samples K instances from a pool of N samples through a uniform distribution, establishing a comparative baseline for subsequent uncertainty or representativeness-based migration strategies. It serves to rigorously evaluate the efficacy of active selection strategies in mitigating distributional shifts.

D. Evaluation Metrics

To comprehensively assess model generalization ability under distribution shift, we employ three complementary evaluation metrics.

Accuracy serves as our primary classification performance metric, measuring the percentage of data being correctly classified. Given a model, predicting the programming problem a code snippet solves, we measure its accuracy on ID and OOD test sets, respectively, in Java250-S, Python800-S and Python75. Generally, a model will have greater accuracy on ID test data than on the OOD since its training data has the same distribution as the ID test set [5].

F1 Score provides a balanced assessment of a classifier's performance, which is particularly crucial for imbalanced datasets like PrimeVul. It is used for vulnerability detection and is dominated by non-vulnerable instances.

Area Under the Receiver Operating Characteristic Curve (AUC-ROC) provides a threshold-independent assessment. Unlike accuracy, which depends on a specific classification threshold, AUC-ROC measures the model's capability to distinguish between different classes across all possible decision thresholds. An ideal OOD detector has the highest AUC-ROC score of 100%.

E. Other Settings

Code model preparation. We train and fine-tune four pre-trained models (RoBERTa, CodeBERT, GraphCodeBERT, StarCoder2-3B) with a learning rate of 0.00005. These four models undergo 5 training epochs with a batch size of 12. The optimization algorithm for all models is the Adam algorithm. Owing to the large parameter scale of StarCoder2-3B, we employ Low-Rank Adaptation (LoRA) for efficient fine-tuning. The LoRA configuration uses a rank of 8 and a scaling factor of

32, targeting the query, key, value, and output projections. Additionally, we verify the absence of data leakage in StarCoder2-3B by randomly selecting 20% of the training dataset and obtaining code similarity (mostly below 30%) through the data source analysis tool provided by Big Code. For other training parameters in CNN and MLP, we follow the default setting in CodeNet. Note that, we cannot train CNN and MLP with acceptable performance (all trained models with random prediction) on the PrimeVul dataset, thus, we did not report these parts of results.

Settings of OOD detectors. For ODIN, we use different perturbation magnitudes (0, 0.0005, 0.001, 0.0014, 0.002, 0.0024, 0.005, 0.01, 0.05, 0.1, 0.2) and set the scaling parameter to 1000. For Mahalanobis, we use different perturbation magnitudes (0, 0.0005, 0.001, 0.0014, 0.002, 0.0024, 0.005, 0.01, 0.05, 0.1, 0.2). We report the best results across these settings. For FLatS, we use the validation sets of ID/OOD data to compute the FLatS scores and calibrate the detection threshold based on the FPR@95 criterion.

Settings of OOD Mitigation. We set the labeling budget as 10% for each OOD mitigation method, which is the maximum number studied by previous works [27], [42], [43].

Experiments Environment. All experiments were conducted on a Linux server equipped with Intel Xeon Silver 4114 CPUs (20 cores, 40 threads) and an NVIDIA Tesla V100 GPU (16GB VRAM). The system utilized 187 GiB RAM and ran Ubuntu 20.04.4 LTS. Software included Python 3.11.7, PyTorch 2.3.0, and CUDA 12.2. The version of the static analysis tool Joern [31] is 4.0.250.

V. RESULTS ANALYSIS

A. RQ1: Generalization Ability of Code Models

First, we utilize CodeS+ to assess the generalization ability of code models. Table V shows the generalization ability of code models under five types of distribution shift introduced by CodeS. Regardless of the dataset and model, the accuracy of the randomly split ID and OOD test sets are similar (the greatest difference is 1.03% in Java250, MLP (Bag)). Considering each model, we observe that the three pre-trained language models (RoBERTa, CodeBERT, and GraphCodeBERT) are more robust against the distribution shift than CNN (Sequence) and MLP (Bag). For example, in Python800-S with token distribution shift, all the pre-trained models can achieve more than 88% accuracy on the OOD test set while CNN (Sequence) and MLP (Bag) only achieve up to 53.43% accuracy. This indicates that pretraining with diverse programming languages is helpful for producing models with better generalization ability on distribution-shifted datasets. For instance, CodeBERT is pre-trained on natural language-programming language pairs in 6 programming languages (Python, Java, JavaScript, PHP, Ruby, and Go). In addition, by comparing the accuracy drop on the OOD test sets, we can draw the conclusion that representation-based distribution shifts are more challenging.

Tables VI and VII summarize the performance of code models under newly introduced distribution shift types: node distribution shift and edge distribution shift, respectively. Meanwhile, given the highly imbalanced nature of the PrimeVul

dataset, we supplement the corresponding F1-score under accuracy, which provides a more reliable performance measure by balancing precision and recall. Across Python75, Java250-S, and Python800-S, all models exhibit catastrophic accuracy drops under both edge and node distribution shifts when transitioning from ID (0–20%) to OOD (20–40%) test sets. Sequence-based models (CNN, MLP) suffer the most dramatic accuracy degradation by 45%–75%. Pretrained models (RoBERTa, CodeBERT, GraphCodeBERT, StarCoder2-3B) show relative resilience, but still suffer from performance degradation (e.g., RoBERTa on Python800-S edge-shift: 49.47%↓ in AST). As a LLM, StarCoder2-3B exhibits a small accuracy drop on OOD tests as evidenced in Tables VI and VII. We also set comprehensive experiments using reversed ID-OOD setups (e.g., training on 40-60% and testing on 0-20% groups). Results consistently revealed symmetric performance degradation, reinforcing that the challenge stems from our designed distribution shifts, which further validate the impact of node/edge distribution shifts. Comparing node distribution shift and edge distribution shift with old types in CodeS, we can see that the newly introduced distribution shift demonstrates higher influence on the performance of code models, indicating the challenge in handling the distribution changes of nodes and edges of program graphs.

Interestingly, compared to other studied tasks, the results on PrimeVul show negligible performance changes on accuracy under distribution shift, e.g., with a maximum accuracy change of 1.5%. This indicates that the distribution shift has a limited impact on the vulnerability detection task. We conjecture this phenomenon can be attributed to two reasons: 1) *Label sparsity and imbalance*. In real-world scenarios, the ratio of positive to negative samples in vulnerability data is heavily imbalanced. Consequently, structural distribution shifts minimally disrupt predictions, as models default to “non-vulnerable” outputs regardless of structural complexity. 2) *Semantic-driven task nature*. Unlike syntax-focused tasks (e.g., code classification), vulnerability detection hinges on semantic signatures rather than structural complexity. Nevertheless, the distribution shifts constructed in CodeS+ still induced a measurable performance degradation, with a maximum F1-score decrease of 9.49% on the PrimeVul dataset. Albeit the impact was relatively mild compared to classification tasks. The varying degrees of performance degradation reveal how different code tasks exhibit distinct vulnerabilities to distribution shifts. This insight underscores the need for task-specific robustness strategies even under the same distribution shift.

Answer to RQ1: Data distribution shift highly affects the prediction correctness of code models, in which node distribution shift and edge distribution shift lead to severe performance degradation.

B. RQ2: Effectiveness of OOD Detectors

Since the data with task distribution shift is real OOD data (these data are not in the classification target of the model), we can use ID/OOD data split based on task distribution shift and random split to evaluate the performance of OOD detectors.

TABLE V
MODEL ACCURACY (%) ON ID AND OOD TEST SETS GIVEN DIFFERENT DNNs AND DISTRIBUTION SHIFTS

Model	Random		Programmer		Time		Token		CST	
	ID Test	OOD Test	ID Test	OOD Test	ID Test	OOD Test	ID Test	OOD Test	ID Test	OOD Test
Python75										
CNN (Sequence)	96.91	97.01 (0.10 ↑)	96.53	96.47 (0.06 ↓)	97.51	92.64 (4.87 ↓)	97.50	61.04 (36.46 ↓)	96.98	71.17 (25.81 ↓)
MLP (Bag)	92.22	92.07 (0.15 ↓)	92.93	91.80 (1.13 ↓)	92.35	88.13 (4.22 ↓)	94.13	45.28 (48.85 ↓)	93.53	63.45 (30.08 ↓)
RoBERTa	98.31	98.13 (0.18 ↓)	98.35	98.46 (0.11 ↑)	97.92	99.28 (1.36 ↑)	98.28	95.02 (3.26 ↓)	99.47	86.65 (12.82 ↓)
CodeBERT	98.37	98.29 (0.08 ↓)	98.30	98.53 (0.23 ↑)	98.06	99.44 (1.38 ↑)	98.51	96.92 (1.59 ↓)	99.52	87.04 (12.48 ↓)
GraphCodeBERT	98.49	98.34 (0.15 ↓)	98.40	98.63 (0.23 ↑)	98.11	99.52 (1.41 ↑)	98.50	96.37 (2.13 ↓)	99.52	87.17 (12.35 ↓)
Java250-S										
CNN (Sequence)	84.23	84.43 (0.20 ↑)	85.86	82.84 (3.02 ↓)	-	-	89.01	65.85 (23.16 ↓)	86.69	75.91 (10.78 ↓)
MLP (Bag)	70.58	71.61 (1.03 ↑)	72.40	71.59 (0.81 ↓)	-	-	76.31	35.32 (40.99 ↓)	76.03	50.86 (25.17 ↓)
RoBERTa	94.87	95.26 (0.39 ↑)	95.53	95.04 (0.49 ↓)	-	-	96.55	87.43 (9.12 ↓)	96.79	87.47 (9.32 ↓)
CodeBERT	95.67	96.10 (0.43 ↑)	96.29	96.25 (0.04 ↓)	-	-	96.92	89.27 (7.65 ↓)	97.41	88.63 (8.78 ↓)
GraphCodeBERT	96.02	96.34 (0.32 ↑)	96.56	96.51 (0.05 ↓)	-	-	97.29	89.89 (7.40 ↓)	97.82	89.31 (8.51 ↓)
Python800-S										
CNN (Sequence)	77.91	78.40 (0.49 ↑)	77.23	66.94 (10.29 ↓)	-	-	79.27	53.83 (25.44 ↓)	77.94	77.82 (0.12 ↓)
MLP (Bag)	66.90	67.00 (0.10 ↑)	67.29	63.39 (3.90 ↓)	-	-	71.37	34.47 (36.90 ↓)	66.39	67.77 (1.38 ↑)
RoBERTa	96.09	96.38 (0.29 ↑)	95.73	96.15 (0.42 ↑)	-	-	97.04	88.51 (8.53 ↓)	95.72	97.31 (1.59 ↑)
CodeBERT	96.48	96.74 (0.26 ↑)	96.19	96.99 (0.80 ↑)	-	-	97.44	89.91 (7.53 ↓)	96.19	97.77 (1.58 ↑)
GraphCodeBERT	96.69	96.89 (0.20 ↑)	96.26	97.24 (0.98 ↑)	-	-	96.19	97.77 (1.58 ↑)	96.42	97.98 (1.56 ↑)

A high AUC-ROC score obtained by the detector indicates that its discrimination ability is strong. Table VIII shows the OOD detection results on the six types of distribution shift originally proposed in CodeS. We can see that the simplest MSP detector produces the best result, where the difference between random and task distribution shifts is larger than by the well-designed Mahalanobis and OE. This phenomenon demonstrates that compared to image and text data, the two programs are more difficult to be distinguished. More powerful OOD detectors are needed.

Tables IX and X present the OOD detection performance under newly supported distribution shift types in CodeS+. The results further demonstrate that existing OOD detection methods cannot distinguish ID and OOD code data. Among the studied detection methods, Mahalanobis demonstrates a more fluctuating detection capability, e.g., the AUC-ROC ranges from 0.11 to 85.6. The potential reason behind this is that Mahalanobis' sensitivity to class density exacerbates its fragility: As the class count increases, the decision boundaries tighten, reducing the discriminative power for OOD detection. This effect is evident in Java250-S, where Mahalanobis AUC-ROC drops to 7.92% (AST node-shift) and 18.71% (CFG node-shift). The simplest method MSP consistently performs well, often surpassing more complex methods like Mahalanobis and OE. The ineffectiveness of CV-based OOD detectors, such as Mahalanobis, stems from a fundamental geometric mismatch between image and code feature spaces. CV models typically learn isotropic, Gaussian-like representations, whereas transformer-based code embeddings exhibit a non-Gaussian, anisotropic structure. Consequently, these detectors derived from CV paradigms become unreliable when applied directly to code representations. FlatS demonstrates strong performance with high AUC-ROC scores, particularly in Python75 where it exceeds 95%. The results can be attributed to the replacement of the public corpus with the OOD validation sets to get the likelihood ratio, which we typically have no access to in the real world. Consequently, while access to data resembling the OOD distribution can

significantly enhance detector capability, the results underscore the fundamental challenge for methods to detect OOD samples without prior information. This systemic fragility underscores that current OOD detectors without prior information, designed primarily for non-structural data, lack intrinsic robustness to edge and node distribution shifts.

Answer to RQ2: Existing OOD detection methods exhibit limitations in identifying OOD samples in CodeS+. The simplest method, MSP, still performs relatively better without prior information. There is an urgent need for OOD detection frameworks explicitly designed for code data and with less prior information.

C. RQ3: Effectiveness of OOD Mitigation Methods

Finally, we investigate the effectiveness of existing retraining-based OOD mitigation methods on enhancing the generalization ability of code models. We compare the model performance before and after retraining. Figs. 5 and 6 illustrate the results on Java250-S, other results can be found on our project site.

Surprisingly, in over half of the cases (23 out of 40), the simple random selection-based method performs the best. This finding is consistent with previous work [27], which states that when the distribution shift is significant, random selection-based enhancement is the best choice. This phenomenon also highlights the crucial need to design effective shift mitigation techniques for code data and models. Moreover, we observe that Core-Set, which is the representative clustering-based method, performs the worst in our setting. The potential reason is that Core-set leverages geometric representativeness in the feature space, maximizing the minimum distance to existing prototypes, so it ensures comprehensive feature-space coverage, which is highly effective for small-scale datasets with fewer labels. However, embeddings of code data are more challenging to learn and Java250-S contains 250 categories. These two

TABLE VIII

AUC-ROC SCORE AND ACCURACY ON DIFFERENT DISTRIBUTION SHIFTS. MODEL: CNN (SEQUENCE). AVERAGE: MEAN OVER 4 OOD DETECTORS

Shift Type	AUC-ROC				Accuracy (%)		
	MSP	ODIN	Mahalanobis	OE	Average	ID Test	OOD Test
Python75							
Random	50.16	62.83	49.07	49.64	52.93	96.91	97.01 (0.10 ↑)
Task	91.33	61.42	70.54	61.11	71.10	96.95	0.00 (0.00)
Programmer	48.20	83.23	50.21	48.33	57.49	96.53	96.47 (0.06 ↓)
Time	57.90	76.80	50.00	50.22	58.73	97.51	92.64 (4.87 ↓)
Token	82.82	71.16	56.70	52.61	65.82	97.50	61.04 (36.46 ↓)
CST	77.89	70.43	57.84	57.84	66.00	96.98	71.17 (25.81 ↓)
Java250-S							
Random	50.06	49.69	52.25	48.12	50.03	84.23	84.43 (0.20 ↑)
Task	80.99	44.85	70.78	52.67	62.32	87.89	0.00 (0.00)
Programmer	53.72	49.36	46.33	49.22	49.66	85.86	82.84 (3.02 ↓)
Token	68.65	52.28	49.47	52.63	55.76	89.01	65.85 (23.16 ↓)
CST	60.32	41.85	50.98	51.53	51.17	86.69	75.91 (10.78 ↓)
Python800-S							
Random	49.67	37.63	48.55	50.00	46.46	77.91	78.40 (0.49 ↑)
Task	78.88	13.13	53.14	50.45	48.90	82.50	0.00 (0.00)
Programmer	58.10	46.34	54.30	62.45	55.30	77.22	66.94 (10.28 ↓)
Token	64.47	39.24	50.52	50.00	51.06	79.27	53.83 (25.44 ↓)
CST	49.74	48.02	48.26	50.00	49.01	77.94	77.82 (0.12 ↓)

TABLE IX

AUC-ROC SCORE AND ACCURACY ON DIFFERENT DISTRIBUTION NODE-SHIFTS. MODEL: CNN (SEQUENCE). AVERAGE: MEAN OVER 5 OOD DETECTORS

Shift Type	AUC-ROC				Accuracy (%)		
	MSP	ODIN	Mah	OE	Average	ID Test	OOD Test
Python75							
Ast	91.66	17.05	75.65	88.68	96.49	73.91	97.50 (30.17 (67.33 ↓))
Cfg	87.78	17.44	61.17	92.26	96.13	70.96	96.92 (30.58 (66.34 ↓))
Dataflow	93.71	22.41	61.80	92.17	96.51	73.32	97.58 (32.83 (64.75 ↓))
Pdg	93.85	28.24	68.50	88.61	96.83	75.21	97.42 (37.25 (60.17 ↓))
Reftype	93.94	20.99	84.24	94.82	96.15	78.03	97.08 (35.50 (61.58 ↓))
Java250-S							
Ast	74.80	25.00	7.92	60.57	83.45	50.35	86.11 (38.64 (47.47 ↓))
Cfg	78.10	21.12	18.71	65.51	83.64	53.42	85.42 (37.88 (47.54 ↓))
Dataflow	79.08	27.25	38.49	71.68	81.58	59.62	84.03 (41.48 (42.55 ↓))
Pdg	75.71	13.04	22.09	60.23	83.30	50.87	82.12 (34.28 (47.84 ↓))
Reftype	77.53	15.03	28.75	69.95	84.47	55.15	84.55 (40.91 (43.64 ↓))
Python800-S							
Ast	71.31	45.90	1.37	58.34	85.07	52.39	64.31 (18.93 (45.38 ↓))
Cfg	72.31	18.03	2.21	61.17	88.45	48.43	65.67 (21.50 (44.17 ↓))
Dataflow	70.38	53.54	0.60	54.04	83.66	52.44	62.23 (19.20 (43.03 ↓))
Pdg	69.65	37.02	4.13	52.67	87.28	50.15	64.31 (24.09 (40.22 ↓))
Reftype	73.54	31.15	0.41	57.46	87.03	49.92	62.32 (20.38 (41.94 ↓))

TABLE X

AUC-ROC SCORE AND ACCURACY ON DIFFERENT DISTRIBUTION EDGE-SHIFTS. MODEL: CNN (SEQUENCE). AVERAGE: MEAN OVER 5 OOD DETECTORS

Shift Type	AUC-ROC				Accuracy (%)		
	MSP	ODIN	Mah	OE	Average	ID Test	OOD Test
Python75							
Ast	94.26	36.09	67.07	84.07	96.67	75.63	97.50 (28.33 (69.17 ↓))
Cfg	91.70	30.16	66.81	90.97	97.51	75.43	97.75 (24.67 (73.08 ↓))
Dataflow	91.20	31.03	67.18	90.56	96.40	75.27	97.08 (32.25 (64.83 ↓))
Pdg	90.79	25.97	54.92	79.89	95.94	69.50	97.08 (31.92 (65.16 ↓))
Reftype	92.00	18.33	85.60	94.25	95.14	77.06	96.83 (30.83 (66.00 ↓))
Java250-S							
Ast	71.74	17.51	25.51	66.58	82.21	52.71	79.69 (35.42 (44.27 ↓))
Cfg	75.04	22.18	41.72	57.63	83.23	55.96	83.85 (36.93 (46.92 ↓))
Dataflow	76.41	23.05	22.95	61.25	75.35	51.80	83.85 (35.59 (48.26 ↓))
Pdg	77.09	27.52	66.68	61.79	83.66	63.35	86.28 (39.58 (46.70 ↓))
Reftype	77.27	22.19	23.42	67.71	71.64	52.45	85.07 (34.09 (50.98 ↓))
Python800-S							
Ast	68.93	40.06	0.44	56.74	84.38	50.11	57.25 (18.75 (38.50 ↓))
Cfg	69.92	35.24	2.38	65.75	84.95	51.65	65.58 (21.56 (44.02 ↓))
Dataflow	69.05	33.41	0.60	56.30	85.81	49.03	64.04 (22.28 (41.76 ↓))
Pdg	65.49	58.76	4.81	46.75	83.79	51.92	60.60 (17.84 (42.76 ↓))
Reftype	66.65	30.76	0.11	52.20	82.89	46.52	62.59 (19.57 (43.02 ↓))

factors limit the effectiveness of Core-Set. In contrast, Entropy and Gini methods, which rely on prediction-level uncertainty, do not suffer from this limitation.

It is worth noting that the phenomenon is significantly amplified in more capable models. In StarCoder2-3B, the other three methods (DeepGini, Entropy, and Core-Set) achieved relative improvements. The Spearman correlation analysis in Fig. 6 provides further insight: StarCoder2-3B exhibits notably high correlation scores (e.g., 0.831 for CFG under Edge), approaching a perfect monotonic relationship where the coefficient would be +1 or -1. This indicates that its stronger representational capacity yields more meaningful uncertainty estimates, which better identify samples near decision boundaries and thus improve the Entropy-based or Gini Coefficient-based method. The Core-Set method still achieves substantial gains, which can be attributed to the enhanced representational capacity afforded by large-scale parameters despite its reliance on the geometric representativeness of embeddings.

Answer to RQ3: Existing OOD mitigation techniques have limited efficacy in enhancing the generalization ability of code models under CodeS+. Designing new and specific OOD mitigation methods for code data is in demand.

VI. DISCUSSION

A. OOD for Large Language Model

In this work, we made a specific consideration for the StarCoder2 model to typically evaluate the generalization ability of large language models. Its transparent data provenance analysis tools allowed us to conclusively exclude our evaluation data from its pre-training corpus, confirming data leakage. Other models are usually pre-trained from large amounts of data (both open-sourced and closed-sourced datasets could be considered). Therefore, it is hard to justify which part of the datasets is in-distribution or out-of-distribution. In contrast, the original works of our studied models, e.g., CodeBERT, clarified their pre-trained data sources. Then, the test samples in CodeS+ can be treated as out-of-distribution data. Our experimental results are designed to validate the substantial impact of node and edge distribution shifts on code models, as demonstrated by the significant performance degradation observed even in robust models like StarCoder2. The primary goal is to elucidate the distribution shift effect itself, rather than to conduct a detailed cross-model comparison. Further investigation is warranted to systematically assess how these shifts manifest across diverse LLM architectures and training paradigms.

B. Threats to Validity

The *internal threat* lies in the implementation of the program analysis framework and code models. We directly employ a mature tool, Joern static analysis tool [31], to extract code graphs, which may introduce selection bias. However, the conclusion is justified by Joern's maturity and demonstrated capability in processing diverse programming languages consistently. By filtering out unparsable samples during preprocessing and ensuring all models are evaluated on graph representations derived



Fig. 5. The effectiveness of sampling-based mitigation techniques under node distribution shift (Dataset: Java250-S).



Fig. 6. The effectiveness of sampling-based mitigation techniques under edge distribution shift (Dataset: Java250-S).

from Joern, we maintain comparability and validity in our performance comparisons, thereby mitigating the impact of this limitation on the experimental outcomes. For the code model construction, we directly use the implementation provided by Hugging Face.

The external threat stems from our studied code tasks, code models, OOD detector, and OOD mitigation techniques. The current version of CodeS+ supports four classification tasks, which have been widely studied by previous works [46], [47]. Besides, our work currently focuses on discriminative tasks, which offer controlled, quantifiable evaluation metrics ideal for systematic distribution shift analysis. While this approach establishes a foundational benchmark, future efforts will extend CodeS+ to include comprehensive generative task datasets to address the evolving landscape of code intelligence. The primary evaluation still relies on programming competition data, which may differ from industrial software in complexity and naming conventions. We have incorporated the PrimeVul dataset to include real-world vulnerability detection scenarios. This represents an initial effort to enhance the practical relevance of our benchmark. Future work will focus on expanding CodeS+ with more controlled, industry-aligned datasets. For code models, both simple MLP and pre-trained code models have been considered in our study. As discussed in Section VI-A, we did not include large language models except StarCoder2 due to the unavailability of pre-trained data. For OOD detectors and OOD mitigation techniques, we have selected the most representative techniques. Moreover, we have open-sourced the datasets and implementation, it is easy to extend other OOD detectors and mitigation methods to evaluate CodeS+. All in all, we believe our studied objectives are representative and diverse enough and the conclusion drawn from this work can be generalized to other similar cases.

VII. RELATED WORK

We review related works from source code learning, distribution shift in source code, and OOD detection three aspects.

A. Source Code Learning

The evolution of source code learning (ML4Code) focuses on leveraging machine learning, particularly deep learning, to extract generalizable features from large-scale source codes. Early approaches like Code2Vec [15] learned distributed code representations by encoding structural paths in abstract syntax trees (ASTs), avoiding direct treatment of code as textual sequences. Meanwhile, Alon et al. [3] proposed Code2Seq, a sequence-to-sequence model adapted from neural machine translation, which decodes path embeddings into target sequences (e.g., method names) rather than generating fixed-dimensional vectors. Recent research demonstrates that integrating structured semantics substantially enhances models' ability to capture program logic. For instance, Program Graphs [48] augment abstract syntax trees (ASTs) with semantic edges, enabling graph neural networks (GNNs) to learn enriched vector representations for improved vulnerability detection. Devign [49] uses graph neural networks to learn the complete semantics

of the program through code property graphs that integrate AST, CFG, and data dependencies. Building on the bimodal foundation of CodeBERT, which jointly learns NL-PL representations through masked language modeling and replaced token detection, GraphCodeBERT introduces data flow graphs to explicitly model value-dependency relationships. Its edge prediction pretraining task and node alignment jointly optimize the understanding of code semantics. Multimodal frameworks like UniXcoder+ [17] unify code, comments, and program dependency graphs (PDGs) [50], highlighting the advantage of multi-source integration for tasks like code search.

Unlike these works on source code learning, our work studies the robustness of various models under dataset distribution shifts. Specifically, we select code classification and vulnerability detection in the downstream tasks of source code learning and introduce distribution transfer challenges to the corresponding datasets.

B. Distribution Shift in Source Code

Research on distribution shifts in source code data has evolved into a systematic framework. The seminal work Wilds [5] first formalized cross-project distribution shift from the problem for code completion in source code learning, revealing how biases across code repositories constrain the generalization of code models. Subsequent empirical studies by Li et al. [25] quantified three dominant distribution shift scenarios in source code tasks: cross-project, cross-developer, and cross-temporal shifts. Critically, their findings revealed that uncertainty metrics prevalent in computer vision (e.g., Monte Carlo Dropout) exhibit significant limitations when applied to source code. This stems from fundamental differences in data modalities between the discrete structural semantics of code and the continuous feature representations of image data. Nie et al. [33] demonstrated that evaluation methodologies themselves can induce distribution shifts. In code summarization, they showed that models trained at time t_0 exhibit performance degradation when applied to an evolved dataset at t_1 due to API deprecation or syntax updates, which can be another distribution shift. Rabin et al. [51] classifies distribution shifts through the lens of learning pathology: pattern-reliance shifts (over-dependence on memorized token sequences) and structural shifts (misalignment in control/data-flow logic between training and deployment). Experiments confirm that over-reliance on memorization amplifies sensitivity to distribution shifts. SIMSCOOD [52] systematically categorizes distribution shifts in fine-tuned models into three types: length-based, syntax-based, and semantic-based distribution shifts. Its analysis reveals that adapter tuning enhances robustness to distribution shifts, and LoRA fine-tuning demonstrates higher OOD generalization accuracy.

While these categorizations address common distribution shift scenarios, they omit a formal characterization of shifts arising from topological complexity in source code semantics. Our work addresses this gap by pioneering graph-based distribution shift quantification via structured parsing of code property graphs (CPGs). We use the number of nodes or edges to

quantify the distribution offset caused by structural differences such as control flow logic and data dependency depth.

C. OOD Detection

For machine language, recent researches address some unique challenges of OOD detection in code models. Rathnasuriya et al. [53] proposed CodeImprove for validation and adaptation of out-of-distribution inputs to become in-distribution inputs for code models. It uses the dropout-based sub-model generation (DSMG) technique to detect OOD samples. LED [54] employs an information-theoretic approach with cluster-contrastive learning and the underlying-data-distribution mechanism to detect OOD code in vulnerable detection. It tries to choose important and label-relevant code statements in each source code data sample during training to boost the data representation learning, facilitating OOD source code data detection. Yan et al. [55] proposed two multi-modal OOD detection approaches for code-related pre-trained ML models, namely unsupervised COOD and weakly-supervised COOD+. The COOD merely leveraged unsupervised contrastive learning to identify OOD samples by aligning comments with code representation. As an extension of COOD, COOD+ combined contrastive learning and a binary classifier for OOD detection using a small number of labeled OOD samples. FLatS introduces a principled OOD detection framework based on the likelihood ratio between out-distribution and in-distribution probabilities. It estimates these densities in the feature space using energy functions approximated by KNN distances, combining both ID and auxiliary OOD data for robust scoring.

Different from proposing new OOD detection methods, we focus on evaluating the effectiveness of existing representative OOD techniques on CodeS+. Our experimental results revealed that those methods, which were originally proposed for image data and NLP data, cannot handle code data properly.

VIII. CONCLUSION

This paper proposed CodeS+, an extension of CodeS, a distribution shift benchmark dataset that supports two programming languages and seven types of distribution shift for code data. Using CodeS+, we evaluated the generalization ability of code models, the effectiveness of OOD detectors, and the usefulness of OOD mitigation techniques. The experimental results revealed that there is a huge gap between the current code models and their practical usage. OOD is a challenging and important issue to be emphasized.

In future, work can be improved from several directions: extending the study of structural distribution shifts to code generation tasks, enriching the characterization of structural complexity beyond node and edge cardinality through more nuanced graph-theoretic measures, and conducting controlled experiments to isolate individual factors (e.g., model architecture, parameter scale) including evaluation of larger models (7B+) to better understand which components contribute most to robustness.

Overall, CodeS+ paves the way for future study of source code learning, including OOD detection, model generalization ability enhancement, and more.

REFERENCES

- [1] L. Mou, G. Li, L. Zhang, T. Wang, and Z. Jin, "Convolutional neural networks over tree structures for programming language processing," in *Proc. 13th AAAI Conf. Artif. Intell.*, 2016, pp. 1287–1293.
- [2] Y. Guo, C. Patsakis, Q. Hu, Q. Tang, and F. Casino, "Outside the comfort zone: Analysing LLM capabilities in software vulnerability detection," in *Proc. Comput. Secur. ESORICS: 29th Eur. Symp. Res. Comput. Secur.*, Bydgoszcz, Poland. Berlin, Heidelberg: Springer-Verlag, 2024, pp. 271–289.
- [3] U. Alon, S. Brody, O. Levy, and E. Yahav, "Code2Seq: Generating sequences from structured representations of code," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–22.
- [4] X. Gu et al., "On the effectiveness of large language models in domain-specific code generation," *ACM Trans. Softw. Eng. Method.*, vol. 34, no. 3, pp. 1–22, Feb. 2025.
- [5] P. W. Koh et al., "Wilds: a benchmark of in-the-wild distribution shifts," in *Proc. 38th Int. Conf. Mach. Learn.*, M. Meila and T. Zhang, Eds., vol. 139, PMLR, Jul. 2021, pp. 5637–5664. [Online]. Available: <https://proceedings.mlr.press/v139/koh21a.html>
- [6] Q. Hu et al., "Codes: Towards code model generalization under distribution shift," in *2023 IEEE/ACM 45th Int. Conf. Softw. Eng.: New Ideas Emerg. Results (ICSE-NIER)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 1–6.
- [7] L. Yang et al., "Out-of-distribution generalization in natural language processing: Past, present, and future," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Assoc. Comput. Linguistics, Dec. 2023, pp. 4533–4559.
- [8] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–16. [Online]. Available: <https://openreview.net/forum?id=HJz6tiCqYm>
- [9] Y. Shi et al., "How to better utilize code graphs in semantic code search?," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2022, pp. 722–733.
- [10] Y. Yan, N. Cooper, K. Moran, G. Bavota, D. Poshvyanyk, and S. Rich, "Enhancing code understanding for impact analysis by combining transformers and program dependence graphs," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 972–995, 2024.
- [11] K. Borowski, B. Balis, and T. Orzechowski, "Semantic code graph—an information model to facilitate software comprehension," *IEEE Access*, vol. 12, pp. 27279–27310, 2024.
- [12] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol.*, Assoc. Comput. Linguistics, 2019, pp. 4171–4186. [Online]. Available: <https://aclanthology.org/N19-1423.pdf>
- [13] M. Hoang, O. A. Bihorac, and J. Rouces, "Aspect-based sentiment analysis using BERT," in *Proc. 22nd Nordic Conf. Comput. linguistics*, 2019, pp. 187–196.
- [14] H. Xu, B. Liu, L. Shu, and P. S. Yu, "BERT post-training for review reading comprehension and aspect-based sentiment analysis," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol.*, 2019, pp. 2324–2335.
- [15] U. Alon, M. Zilberstein, O. Levy, and E. Yahav, "code2vec: Learning distributed representations of code," *Proc. ACM Program. Lang.*, vol. 3, no. POPL, pp. 1–29, 2019.
- [16] W. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol.*, K. Toutanova, A. Rumshisky, and L. Zettlemoyer, Eds., Assoc. Comput. Linguistics, Jun. 2021, pp. 2655–2668.
- [17] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, 2022, pp. 7212–7225.
- [18] C. Niu, C. Li, V. Ng, J. Ge, L. Huang, and B. Luo, "SPT-code: Sequence-to-sequence pre-training for learning source code representations," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 2006–2018.
- [19] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and

- generation,” in *Proc. Conf. Empirical Methods in Natural Lang. Process.* M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, Eds., Punta Cana, Dominican Republic: Assoc. for Comput. Linguistics, Nov. 2021, pp. 8696–8708.
- [20] Z. Feng et al., “CodeBERT: A pre-trained model for programming and natural languages,” in *Findings Assoc. Comput. Linguistics: EMNLP*, Assoc. for Comput. Linguistics, Nov. 2020, pp. 1536–1547. [Online]. Available: <https://aclanthology.org/2020.findings-emnlp.139>
- [21] D. Guo et al., “GraphCodeBERT: Pre-training code representations with data flow,” in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–18. [Online]. Available: <https://openreview.net/forum?id=jLoC4ez43PZ>
- [22] S. Liu, X. Xie, J. Siow, L. Ma, G. Meng, and Y. Liu, “GraphSearchNet: Enhancing GNNs via capturing global dependencies for semantic code search,” *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 2839–2855, Apr. 2023.
- [23] Y. Liu et al., “RoBERTa: A robustly optimized BERT pretraining approach,” 2020. [Online]. Available: <https://openreview.net/forum?id=SyxS0T4tvS>
- [24] R. Puri et al., “CodeNet: A large-scale ai for code dataset for learning a diversity of coding tasks,” in *35th Conf. Neural Information Process. Syst. (NeurIPS) Track Datasets Benchmarks (NIPS)*, Red Hook, NY, USA: Curran Associates Inc., 2021, pp. 1–22.
- [25] Y. Li, S. Chen, and W. Yang, “Estimating predictive uncertainty under program data distribution shift,” 2021, *arXiv:2107.10989*.
- [26] Z. Li, X. Ma, C. Xu, C. Cao, J. Xu, and J. Lü, “Boosting operational DNN testing efficiency through conditioning,” in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, New York, NY, USA: ACM, 2019, pp. 499–509.
- [27] Q. Hu et al., “An empirical study on data distribution-aware test selection for deep learning enhancement,” *ACM Trans. Softw. Eng. Method.*, vol. 31, no. 4, pp. 78:1–78:30, 2022, doi: 10.1145/3511598.
- [28] T. Sharma, M. Kechagia, S. Georgiou, R. Tiwari, and F. Sarro, “A survey on machine learning techniques for source code analysis,” 2021, *arXiv:2110.09610*.
- [29] “Atcoder,” 2012. Accessed: May 19, 2022. [Online]. Available: <https://atcoder.jp/>
- [30] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, “Modeling and discovering vulnerabilities with code property graphs,” in *Proc. IEEE Symp. Secur. Privacy*, Piscataway, NJ, USA: IEEE Press, 2014, pp. 590–604.
- [31] Joern, “The bug hunter’s workbook,” 2025. Accessed: Jul. 30, 2022. [Online]. Available: <https://joern.io/>
- [32] Y. Ding et al., “Vulnerability detection with code language models: How far are we?,” in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng. (ICSE)*, Los Alamitos, CA, USA: IEEE Computer Society., May 2025, pp. 1729–1741, doi: 10.1109/ICSE55347.2025.00038.
- [33] P. Nie, J. Zhang, J. J. Li, R. J. Mooney, and M. Gligoric, “Impact of evaluation methodologies on code summarization,” in *Proc. 60th Annu. Meet. Assoc. Comput. Linguistics (Volume 1: Long Papers)*, Dublin, Ireland, 2022, pp. 4936–4960.
- [34] S. Lu et al., “CodeXGLUE: A machine learning benchmark dataset for code understanding and generation,” in *Proc. 35th Conf. Neural Inf. Process. Syst. Datasets Benchmarks Track*, vol. Round 1, 2021, pp. 1–16. [Online]. Available: <https://openreview.net/forum?id=6IE4dQXaUcb>
- [35] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, “StarCoder 2 and the stack V2: The next generation,” 2024.
- [36] S. Fort, J. Ren, and B. Lakshminarayanan, “Exploring the limits of out-of-distribution detection,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 7068–7081.
- [37] D. Hendrycks and K. Gimpel, “A baseline for detecting misclassified and out-of-distribution examples in neural networks,” in *Proc. 5th Int. Conf. Learn. Representations (ICLR)*, Toulon, France, 2017. [Online]. Available <https://openreview.net/forum?id=Hkg4T19xl>
- [38] S. Liang, Y. Li, and R. Srikant, “Enhancing the reliability of out-of-distribution image detection in neural networks,” in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–15. [Online]. Available: <https://openreview.net/forum?id=H1VGkIxRZ>
- [39] K. Lee, K. Lee, H. Lee, and J. Shin, “A simple unified framework for detecting out-of-distribution samples and adversarial attacks,” in *Proc. 32nd Int. Conf. Neural Inform. Process. Syst. (NIPS)*, Red Hook, NY, USA: Curran Associates Inc., 2018, pp. 7167–7177.
- [40] D. Hendrycks, M. Mazeika, and T. Dietterich, “Deep anomaly detection with outlier exposure,” in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–18. [Online]. Available: <https://openreview.net/forum?id=HyxCxxReY7>
- [41] H. Lin and Y. Gu, “Flats: Principled out-of-distribution detection with feature-based likelihood ratio score,” in *Proc. Empirical Methods Natural Lang. Process. (EMNLP)*, 2023.
- [42] Y. Feng, Q. Shi, X. Gao, J. Wan, C. Fang, and Z. Chen, “DeepGini: prioritizing massive tests to enhance the robustness of deep neural networks,” in *Proc. 29th ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2020, pp. 177–188.
- [43] W. Shen, Y. Li, L. Chen, Y. Han, Y. Zhou, and B. Xu, “Multiple-boundary clustering and prioritization to promote neural network re-training,” in *Proc. 35th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2020, pp. 410–422.
- [44] D. Wang and Y. Shang, “A new active labeling method for deep learning,” in *Proc. Int. joint Conf. Neural Netw. (IJCNN)*, Piscataway, NJ, USA: IEEE Press, 2014, pp. 112–119.
- [45] O. Sener and S. Savarese, “Active learning for convolutional neural networks: A core-set approach,” in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–13. [Online]. Available: <https://openreview.net/forum?id=H1aIuk-RW>
- [46] Z. Yang, J. Shi, J. He, and D. Lo, “Natural attack for pre-trained models of code,” in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 1482–1493.
- [47] Z. Tian, J. Chen, and Z. Jin, “Code difference guided adversarial example generation for deep code models,” in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 850–862.
- [48] M. Allamanis, M. Brockschmidt, and M. Khademi, “Learning to represent programs with graphs,” 2017, *arXiv:1711.00740*.
- [49] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, “Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks,” in *Proc. 33rd Int. Conf. Neural Inform. Process. Syst.*, Red Hook, NY, USA: Curran Associates Inc., December 2019, pp. 10197–10207. [Online]. Available: <https://dl.acm.org/doi/pdf/10.5555/3454287.3455202>
- [50] R. V. Chen, “Case study of feature location using dependence graph,” in *Proc. 8th Int. Workshop Program Comprehension*, Piscataway, NJ, USA: IEEE Press, 2000, pp. 241–247.
- [51] M. R. I. Rabin, A. Hussain, M. A. Alipour, and V. J. Hellendoorn, “Memorization and generalization in neural code intelligence models,” *Inf. Softw. Technol.*, vol. 153, 2023, art. no. 107066.
- [52] H. Hajipour, N. Yu, C.-A. Staicu, and M. Fritz, “SimSCOOD: Systematic analysis of out-of-distribution generalization in fine-tuned source code models,” in *Proc. Findings Assoc. Comput. Linguistics (NAACL)*, 2024, pp. 1400–1416.
- [53] R. Rathnasuriya, Z. Zhao, and W. Yang, “CodeImprove: Program adaptation for deep code models,” in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng. (ICSE)*, Los Alamitos, CA, USA: IEEE Comput. Soc. Press, 2025, pp. 676–676.
- [54] V. Nguyen, X. Yuan, T. Wu, S. Nepal, M. Grobler, and C. Rudolph, “Deep learning-based out-of-distribution source code data identification: How far have we gone?” 2024, *arXiv:2404.05964*.
- [55] Y. Yan, V. Duong, H. Shao, and D. Poshyanyk, “Towards more trustworthy deep code models by enabling out-of-distribution detection,” in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng. (ICSE)*, Los Alamitos, CA, USA: IEEE Comput. Soc. Press, 2025, p. 714.